

Lesson 2 — Drawing With Turtle

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

Setup (copy into every turtle program)

```
import turtle

screen = turtle.Screen()
screen.setup(width=800, height=600)
screen.bgcolor("midnightblue")
screen.tracer(0)          # stop auto-
                             redrawing

t = turtle.Turtle()
t.hideturtle()
t.speed(0)

# ... drawing ...

screen.update()           # NOW show it
screen.exitonclick()      # keep the window
open
```

Turtle's coordinates are the odd ones out

	Turtle	Everything else
(0,0)	the middle	top-left
y grows	up	down

Always ask which convention you are in.

Placing, not walking

```
t.penup()          # lift the pen
t.goto(x, y)       # jump
t.pendown()        # pen back down
```

Forget `penup` and lines connect everything.

A square

```
for _ in range(4):
    t.forward(size)
    t.left(90)
```

Four 90° turns = 360°, a full circle. Any polygon:

`angle = 360 / sides`

Grid to screen

```
LEFT = -(COLUMNS * CELL) / 2 + CELL / 2
TOP  = (ROWS * CELL) / 2 - CELL / 2

def cell_to_pixels(column, row):
    return LEFT + column * CELL, TOP - row *
    CELL
```

Minus on the y, because rows go down and turtle's y goes up.

Why `tracer(0)` / `update()`

Draw the whole frame invisibly, then show it in one go. That is **double buffering**, and every graphics system does it. Without it you watch the picture being assembled.

Words

Sprite — one drawn thing. **Heading** — which way the turtle faces. **Origin** — where (0,0) is.

Section A — Recall

A1. Where is `(0, 0)` in turtle, and which way does y grow? How is that different from tkinter and the web canvas? [3 marks]

A2. What do `screen.tracer(0)` and `screen.update()` each do? Why do you need both? [2 marks]

A3. What is the `penup()` / `goto()` / `pendown()` pattern for? [2 marks]

A4. To draw a regular hexagon (6 sides), how many degrees should the turtle turn each time? Show your working, then write the loop. [3 marks]

A5. Give two reasons — other than saving typing — to write `draw_sprite()` rather than repeating the drawing code. [3 marks]

Section B — Predict the output

B1. The grid has 8 columns, `CELL` is 50. Work out `LEFT`, then the x of the centre of column 0 and column 7. [3 marks]

B2. What appears on the screen? Be precise — this is not a trick about the drawing commands. [4 marks]

```
screen.tracer(0)
draw_board()
screen.exitonclick()
```

B3. What shape does this draw, and where does the turtle end up facing?

[3 marks]

```
for _ in range(5):  
    t.forward(80)  
    t.left(72)
```

B4. Using `cell_to_pixels` with `TOP = 150` and `CELL = 50`, what is the y of row 0, row 1 and row 4? [4 marks]
Which is highest on the screen?

Section C — Find and fix the bug

C1. The program runs, draws nothing visible, and the window closes instantly. Two separate things are [3 marks]
wrong. Name both.

```
screen = turtle.Screen()  
screen.tracer(0)  
draw_board()
```

C2. The sprites appear in the right places, but ugly lines connect every one of them to the next. What is [3 marks]
missing, and where?

```
def draw_sprite(x, y, size, colour):  
    t.goto(x, y)  
    t.color(colour)  
    t.begin_fill()  
    for _ in range(4):  
        t.forward(size)  
        t.left(90)  
    t.end_fill()
```

C3. The board is drawn upside down: the top row of the `BOARD` list appears at the bottom of the [4 marks]
window. What single character is wrong, and why does it have that effect?

```
def cell_to_pixels(column, row):  
    return LEFT + column * CELL, TOP + row * CELL
```

Section D — Build it

- ☐ **Checkpoint 1.** Get a window open with the setup boilerplate, and draw one square anywhere. Make sure the window stays open.
- ☐ **Checkpoint 2.** Write `draw_sprite(x, y, size, colour)` that draws a square **centred** at `(x, y)`. Test it by drawing one at `(0, 0)` and checking it is in the middle.
- ☐ **Checkpoint 3.** Use a loop to draw a grid of squares. Get them evenly spaced and centred in the window.
- ☐ **Checkpoint 4.** Write `cell_to_pixels(column, row)` and use it, so the grid code works in *columns and rows* rather than pixels.
- ☐ **Checkpoint 5.** Add a `BOARD` list of strings and draw different things for different characters. Change the board and reload to prove the drawing code did not need touching.
- ☐ **Checkpoint 6.** Design and draw your own sprite — a spaceship, a cat, your initials — as a function taking `(x, y, size)`, and put it on the board.

IF YOU GET A BLANK WINDOW

You have `tracer(0)` but no `update()`. Every drawing command ran perfectly

and the result was never shown. Go looking at the end of your program, not at the drawing code.

If the window flashes open and shuts, you are missing `screen.exitonclick()`.

Section E — Design challenge

NO RIGHT ANSWER

E1. `draw_sprite` currently takes five arguments. **How many is too many?** What would you do if it also needed a border, a rotation, an opacity and a text label?

E2. Your board draws from a list of strings, one character per cell. What would you change if a cell needed to hold *two* things — a floor tile *and* an item standing on it?

E3. Drawing 48 squares takes a moment. Drawing 4,800 would be slow enough to notice. Without knowing how turtle works inside, suggest **two different** ways to make it faster. *Hint: does everything need redrawing every time?*

E4. Turtle puts `(0,0)` in the middle; tkinter puts it in the corner. If you had to work in both at once, what would you write to stop yourself making mistakes? (You will need this in lesson 5.)

Stretch goals

1. Draw your own sprite as a reusable function.
2. A chequerboard — `(row + column) % 2` is 0 or 1.
3. `draw_polygon(sides, length)` . Try 3, 5, 8, then 36. When does it stop looking like a polygon?
4. A spiral: move forward a little more each step. Then turn **89°** instead of 90° and see what happens.
5. Time your board with and without `tracer(0)` using `time.time()` . The ratio will surprise you.